

Weekly Project Progress Report

Team Name: Quick Bite

Week Number: Week 8 – Advanced Client-Side Features (AJAX)

Team Members:

1. Abdullah Mohammed AbuZaid	120211559
2. Abdullah Ali Al-Hindawi	120211863
3. Abdalkareem Rajab Abo Younis	120211877
4. Hazem Mohamed Oukal	120212771

GitHub Repository: <https://github.com/AbdullahAbuZaid04/quick-bite-restaurant.git>

Work completed this week

This week we started bootstrapping the Backend service for the Quick Bite restaurant ordering system. The backend is a work in progress and will be completed in the coming weeks.

The service is built with Node.js and Express on top of MySQL, and follows a layered architecture (routes → middleware → controllers → database) so each concern lives in its own module and can be tested independently.

The main pieces we began working on this week:

1. Database layer. We created the InnoDB schema with seven tables: users, categories, menu_items, orders, order_items, payments, and invoices. We also added MySQL triggers that automatically compute order_items.subtotal on insert/update, and re-aggregate orders.total_amount whenever items are added, edited, or removed.
2. REST API. Started building versioned endpoints under /api covering authentication, category and menu management, order placement, payments, invoices, user management, and an admin dashboard. All responses use a single consistent envelope { success, data, meta } or { success: false, error }.
3. Authentication and authorization. Passwords are stored as bcrypt hashes. On successful login we issue a JSON Web Token. Two middleware helpers — authenticate and authorize(...roles) — protect each route, with four roles supported: customer, admin, manager, and courier.
4. Order workflow. Order creation runs inside a single MySQL transaction: we lock the relevant menu rows (SELECT ... FOR UPDATE), verify they exist and are available, copy the current price into unit_price, and let the trigger fill in subtotals/totals. If any item fails validation the whole transaction is rolled back.

5. Payments and invoices. When a payment moves to paid, the service auto-confirms the order if it was still pending and auto-issues an invoice. Both side-effects happen inside the same transaction as the payment update.
6. Security and reliability. Added helmet, CORS allow-list, rate limiting (300 req/min per IP), parameterised SQL, and a centralised error handler that translates MySQL errors into clean 4xx responses.
7. Testing. A smoke test with 17 assertions, all passing.

Features implemented

- POST /api/auth/register, POST /api/auth/login, GET /api/auth/me, PATCH /api/auth/me
- Categories CRUD: GET/POST/PUT/DELETE /api/categories
- Menu items CRUD + search/filter + availability toggle
- Orders: POST /api/orders (transactional), list, get, cancel, PATCH /api/orders/:id/status
- Payments: POST /api/payments, PATCH /api/payments/:id, GET /api/orders/:id/payments
- Invoices: auto-issued on first successful payment
- User management for admins: list, view, update role, delete
- Admin dashboard: counters, orders-by-status, today/total revenue, recent orders
- GET /api/health for uptime probes

Problems encountered

- Order creation needed to read prices from the database and prevent racing on the same menu item. A naive read-then-insert would leave a TOCTOU gap.
- MySQL foreign-key errors surfaced as raw codes like ER_ROW_IS_REFERENCED_2, which are useless to API consumers.
- Joi schemas were initially duplicated across each controller, making validation rules drift.

Solutions applied

Solutions applied- Order creation runs inside SELECT ... FOR UPDATE transaction to lock rows and prevent race conditions- A single errorHandler middleware maps MySQL error codes to friendly 4xx responses- All Joi schemas lifted into src/validators/ with a single validate(schema) middleware

Plan for next week

- Add image upload endpoint for menu_items.image_url (multer)
- Add server-side pagination and sorting to orders list endpoint
- Add refund flow with payment status transition paid → refunded
- Begin containerisation with Docker and prepare deployment target (Render / Railway / VPS)

Questions for instructor

- Should the database SQL dump (with sample data) be committed to the repository for grading, or only the schema file?
- For deployment, would you prefer we host on a free PaaS (Render / Railway) or document a local-only setup?